

Ativ4 - CP6

Prof. Edson de Oliveira



Orientações:

- Metade da pontuação será atribuída pelo visto na aula quando a aula for presencial, se for remota só vale 1 pt da entrega
 - ✓ A outra metade será dada pela entrega no TEAMS considerando o atendimento às Orientações, Entregas e Requisitos da versão.
- Esta é uma atividade evolutiva, ou seja, a cada semana você deverá atualizar e aprimorar sua versão.
- As **entregas parciais** devem ser feitas pelo Teams, na tarefa indicada para cada etapa.
 - ✓ Cada **entrega parcial** será avaliada conforme o atendimento aos requisitos solicitados.
- A **entrega final** será feita na última semana de aula (data que informarei depois).
 - ✓ Na entrega final, a pontuação será baseada nos acertos e erros do seu trabalho.

Entregas:

- Este projeto pode ser realizado em grupos de até 3 pessoas.
- Cada integrante deve enviar sua própria cópia dos códigos pelo Teams.
- Inclua o nome completo de todos os integrantes (um nome em cada linha) em forma de comentário nas primeiras linhas do(s) arquivo(s) .py.
- Utilize apenas os recursos apresentados em aula. Sujeito a perda de pontos.
- Se utilizar ferramentas de IA, zerará a pontuação..

Descrição da atividade:

Esta aplicação representa a primeira versão do nosso projeto de fechamento do ano.

A cada nova etapa, será solicitada uma evolução da versão anterior, até chegarmos à versão final do projeto.

Versão 1:

- Pense em um tema para criar uma tabela no banco de dados.
- Monte a tabela com campos relacionados a esse tema, pois ela será utilizada e aprimorada até o final do projeto.
- Crie uma tabela com pelo menos 6 campos (colunas) e defina uma chave primária para identificar de forma única cada registro.
 - ➔ Crie pelo menos um campo de cada um dos seguintes tipos de dado: string, int, float e data.
- Desenvolva o itens abaixo utilizando a tabela que você desenvolveu.

<TEMA>

```
0 - Sair
1 - Cadastrar <tema>
2 - Pesquisar <tema>
3 - listar todos os registros <tema>
```

Escolha: _

Requisitos:

- Cadastre somente registros cuja chave primária ainda não exista no banco de dados.
- Pesquise os registros utilizando a chave primária.
- Liste todos os registros que estiverem cadastrados na tabela.

Ativ4 - CP6

Prof. Edson de Oliveira

Versão 2:

- Efetuar a consistência dos dados (verificar se ele é válido para o seu tipo) no lado Python, para que na tabela do banco de dados não ocorram falhas na gravação por causa dos tipos inválidos.
- Acrescentar as opções **Editar** e **Apagar** no menu, tomando como referência a chave primária para conte de pesquisa. Lembre-se de antes de Editar ou Apagar **apresentar o registro** para que o usuário valide a operação:

```
<TEMA>

0 - Sair
1 - Cadastrar <tema>
2 - Pesquisar <tema>
3 - listar todos os registros <tema>
4 - Editar registro <tema>
5 - Apagar registro <tema>

Escolha: _
```

Para quem estiver com dificuldades em converter os valores no Python antes de inseri-los na tabela, segue um exemplo de código:

```
1 import oracledb
2 from datetime import datetime
3
4 # Configuração da conexão – ajuste conforme seu ambiente
5 dsn = oracledb.makedsn("localhost", 1521, service_name="xe")
6 conn = oracledb.connect(user="usuario", password="senha", dsn=dsn)
7
8 # Criação do cursor
9 cur = conn.cursor()
10
11 # Exemplo de dados – poderiam vir de input(), CSV, etc.
12 numero = int(input("Número do aluno: "))
13 nome = input("Nome do aluno: ")
14 nota = float(input("Nota: "))
15 data_str = input("Data de nascimento (dd/mm/aaaa): ")
16
17 # Conversão da data de string para datetime
18 data_nascimento = datetime.strptime(data_str, "%d/%m/%Y")
19
20 # Comando SQL parametrizado (evita SQL Injection)
21 sql = """
22 INSERT INTO aluno (numero, nome, nota, data_nascimento)
23 VALUES (:1, :2, :3, :4)
24 """
25
26 # Execução do comando
27 cur.execute(sql, (numero, nome, nota, data_nascimento))
28
29 # Confirma a transação
30 conn.commit()
31
32 print("Registro inserido com sucesso!")
33
34 # Fecha recursos
35 cur.close()
36 conn.close()
```

Considere uma tabela chamada aluno com os campos: numero (integer), nome (varchar2), nota (float) e data_nascimento (date) no Oracle.

No Python, os dados são digitados e convertidos com o devido casting. Observe que a variável data_str é lida como uma string.

Conforme visto em aula, na linha 18, essa string é convertida em um objeto de data pelo método strptime(), atribuindo o valor convertido à nova variável data_nascimento.

Com as variáveis devidamente convertidas, utilizamos parâmetros posicionais (linha 23), que empregam os marcadores :1, :2, :3 e :4 para representar, na ordem, os valores correspondentes das variáveis.

Dessa forma, a sintaxe do método .execute() muda: além da string SQL, passamos também uma tupla com as variáveis na mesma ordem dos marcadores.

O uso de marcadores torna o processo de inserção mais seguro e confiável, evitando falhas de conversão e possíveis vulnerabilidades no código.

Ativ4 - CP6

Prof. Edson de Oliveira

Inicialmente poderíamos pensar na concatenação de strings e criar a string sql desta forma:

```
sql = f"INSERT INTO aluno VALUES ({numero}, '{nome}', {nota}, TO_DATE('{data_nascimento}', 'DD/MM/YYYY'))"
cur.execute(sql)
```

No entanto, essa forma de concatenação pode gerar vulnerabilidade a um SQL Injection — um tipo de ataque em que comandos maliciosos são inseridos dentro da string SQL.

Ao utilizar parâmetros posicionais, evitamos esse tipo de problema, pois a instrução é executada de forma segura.

Segue o exemplo:

```
sql = "INSERT INTO aluno (numero, nome, nota, data_nascimento) VALUES (:1, :2, :3, :4)"
cur.execute(sql, (numero, nome, nota, data_nascimento))
```

Como alternativa você pode utilizar os parâmetros nomeados em vez de posicionais.

Segue o exemplo:

```
sql = """
INSERT INTO aluno (numero, nome, nota, data_nascimento)
VALUES (:numero, :nome, :nota, :data_nascimento)
"""
cur.execute(sql, {
    "numero": numero,
    "nome": nome,
    "nota": nota,
    "data_nascimento": data_nascimento
})
```

Sabendo destas possibilidades, use a que melhor te convier.

Ativ4 - CP6

Prof. Edson de Oliveira

Versão 3:

- Modifique o menu “3 - Listar todos os registros <tema>” para “3 - Listar Registros <tema>”.
- No submenu do item 3 aparecerá:
 - “a - Todos” —> Terá o mesmo efeito do antigo 3
 - “b - <Pesquisar por parte da string e listar>” —> Escolha um **campo str** da sua tabela para pesquisa. O usuário digitará parte da pesquisa e a pesquisa 0,N será listada.
 - “c - <Pesquisar por um campo numérico e listar>” —> Escolha um campo numérico da sua tabela e aplique os operadores relacionais (>, >=, <, <=, == ou !=) na pesquisa, listando os registros se encontrados. Monte o submenu com os operadores relacionais da forma que desejar.

```
<TEMA>

0 - Sair
1 - Cadastrar <tema>
2 - Pesquisar <tema>
3 - Listar Registros <tema>
    a - Todos
    b - <Pesquisar por parte da String e listar>
    c - <Pesquisar por um campo numérico e listar>
4 - Editar Registros <tema>
5 - Apagar registros <tema>

Escolha: _
```

```
3 - a. Listar todos os registros
3 - b. Nome: Eds
      NOME          XXX   YYY   ZZZ
      Edson de Oliveira  xxxx  xxxx  xxxx
      Edson Vieira      xxxx  xxxx  xxxx
      José Edson        xxxx  xxxx  xxxx
3 - c. Idade: 40
      filtro (>, >=, <, <=, == ou !=): >
      NOME          Idade YYY   ZZZ
      Edson de Oliveira  41    xxxx  xxxx
      Manoel Vieira     60    xxxx  xxxx
      José Silva        70    xxxx  xxxx
```

```
**
SELECT * FROM clientes WHERE nome LIKE 'J%';
SELECT * FROM produtos WHERE nome LIKE '%o';
SELECT * FROM despesas WHERE descricao LIKE '%escola%';
SELECT * FROM funcionarios WHERE nome LIKE '_a%';
SELECT * FROM funcionarios WHERE idade > 40;
```

```
valor = int(input("Idade: ") # 40
```

Ativ4 - CP6

Prof. Edson de Oliveira

```
operador = input("filtro (>, >=, <, <=, == ou !=):") # >=
sql = f"SELECT * FROM funcionarios WHERE IDADE {operador} {valor}"
```

Versão 4:

- No item **"3 - Listar Registros <tema>"** e todos os seus subitens, depois de listar os registros dê a opção de gerar uma Planilha ou arquivo CSV.
- Para tal pergunte: **"Gerar arquivo [E]xcel, [C]SV? Ou [ENTER] para voltar ao menu."**
- Caso a escolha seja por gerar um arquivo, solicite ao usuário um Nome para o arquivo e a extensão deve ser incorporada automaticamente.

<TEMA>

```
0 - Sair
1 - Cadastrar <tema>
2 - Pesquisar <tema>
3 - Listar Registros <tema>
    a - Todos
    b - <Pesquisar por parte da String e listar>
    c - <Pesquisar por um campo numérico e listar>
    Gerar arquivo [E]xcel, [C]SV? Ou [ENTER] para voltar ao menu.
4 - Editar Registros <tema>
5 - Apagar registros <tema>
```

Escolha: _

Versão 5:

Nesta versão criaremos no **item 3** o subitem **"d - Pesquisa genérica"** onde parte de um texto digitado pelo usuário será base de uma pesquisa automática ao menos 3 campos strings, retornando um Dataframe com todos os registros que contenham esta parte da string.

<TEMA>

```
0 - Sair
1 - Cadastrar <tema>
2 - Pesquisar <tema>
3 - Listar Registros <tema>
    a - Todos
    b - <Pesquisar por parte da String e listar>
    c - <Pesquisar por um campo numérico e listar>
    d - <pesquisa genérica>
    Gerar arquivo [E]xcel, [C]SV? Ou [ENTER] para voltar ao menu.
4 - Editar Registros <tema>
5 - Apagar registros <tema>
```

Escolha: _

Ativ4 - CP6

Prof. Edson de Oliveira

Versão 6:

Na opção 3, independentemente da escolha feita no submenu, após selecionar o filtro de pesquisa, devem ser listados todos os campos da tabela.

Com os campos exibidos, o usuário poderá selecionar quais colunas deseja visualizar na listagem dos registros retornados.

A forma de implementação e a usabilidade dessa funcionalidade ficam a critério do grupo.

<TEMA>

- 0 - Sair
- 1 - Cadastrar <tema>
- 2 - Pesquisar <tema>
- 3 - Listar Registros <tema>

a - Todos

b - <Pesquisar por parte da String e listar>

c - <Pesquisar por um campo numérico e listar>

d - <pesquisa genérica>

Gerar arquivo [E]xcel, [C]SV? Ou [ENTER] para voltar ao menu.

- 4 - Editar Registros <tema>
- 5 - Apagar registros <tema>

Escolha: _